Exercise 1 — Producer/Consumer: Random Number

Create two threads: - The first thread: after 5 seconds, generates a random number from 1 to 10 and displays it on screen. - The second thread: receives the number produced by the first thread and displays the corresponding word.

Example:

```
3
three
1
one
5
five
...
```

Write a `main()` method with an infinite loop to demo these two threads. When the first thread produces the number 10, the program exits. You must synchronize the two threads above.

Exercise 2 — Sum of an Array Using a Thread

Write a program that uses a thread to calculate the sum of the elements in an integer array.

Exercise 3 — Printing Numbers in Random Order Using a Thread

Write a program that uses a thread to print the integers from 1 to 10 in a random order.

Exercise 4 — Countdown Timer Using a Thread

Write a program that uses a thread to implement a countdown from 10 down to 1.

Exercise 5 — Queue Operations Using a Thread

Write a program that uses a thread to create a queue and perform add/remove operations on it.

Exercise 6 — Elapsed Time Counter Using a Thread

Write a program that uses a thread to implement a timer that prints the elapsed time.

Exercise 7 — Two Threads Modifying a Shared Array

Create a program with two threads, `Thread1` and `Thread2`.

- `Thread1` stores the array: `int[] numList = {1, 3, 4, 6, 7};`
- `Thread2` adds 10 to each element of the array from `Thread1`.

Create a `Main` class to run the two threads and print the result to the screen.

Exercise 8 — Odd/Even Numbers Printed in Order by Two Threads

Write a program with 2 threads: - The first thread prints the odd numbers from 1 to n. - The second thread prints the even numbers from 0 to n.

n is a positive integer entered from the keyboard. The numbers must appear on the console in natural numeric order.

Example: For n = 8:

```
Even thread: 0
Odd thread: 1
Even thread: 2
Odd thread: 3
Even thread: 4
Odd thread: 5
Even thread: 6
Odd thread: 7
Even thread: 8
Done
```

Exercise 9 — Alternating Uppercase/Lowercase Letters by Two Threads

Write a program with 2 threads: - The first thread prints the uppercase letters from A to Z. - The second thread prints the lowercase letters from a to z.

The letters must appear on the console interleaved in alphabetical order (A, a, B, b, C, c, ... Z, z), then the program prints "Done".

Exercise 10 — Day-of-the-Week Producer/Consumer (Two Languages)

Create 2 threads to do the following task:

- The first thread: after one second, produces a random day of the week and displays it (in Vietnamese, e.g. "Thu hai", "Thu ba", "Thu tu", "Thu nam", "Thu sau", "Thu bay", "Chu nhat").
- The second thread: receives the random day produced by the first thread and displays the corresponding day name in English (e.g. "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday").

Write a `main()` method to demonstrate these threads. You must synchronize the two threads above.

Exercise 11 — Bank Deposit/Withdrawal Threads

Create two threads: 1. **First thread:** simulates depositing money into a shared bank account. Every 3 seconds, it deposits a random amount between 100 and 1000 (currency units). After each deposit, it displays the total balance. 2. **Second thread:** simulates withdrawing money from the shared account. Every 4 seconds, it tries to withdraw a random amount between 100

and 500. After each withdrawal, it checks whether the balance is sufficient and displays the remaining balance, or an error message if the withdrawal would exceed the balance.

Synchronization requirement: ensure that both threads properly access and modify the shared balance to avoid race conditions.

Example output:

```
Thread 1 deposits 500, total balance is 500.
Thread 2 tries to withdraw 300, total balance is 200.
```

Exercise 12 — Word Uppercase Producer/Consumer

Create a class to store an array of strings: `String[] words = {"Sai Gon", "Da Nang", "Hue"};`

Create a thread class that takes each string from the array, converts it to uppercase, and passes the converted string to a second thread class for display.

Create a display thread class that displays the uppercase string received from the first thread. For example, when the first thread sends “Sai Gon”, the display thread should show “SAI GON”.

Create a test class to demonstrate the two thread classes above.

Exercise 13 — String Reversal Producer/Consumer

Create a class to store the following array of strings: `String[] data = {"Hello", "Java", "Programming"};`

Create a thread class that takes each string from the array and reverses it, then sends the reversed value to a second thread class for display.

Create a second thread class that displays the reversed strings received from the first thread.

Create a test class to demonstrate the two thread classes above.

Note: use `synchronized`, `wait`, and `notify` to coordinate the data hand-off between the threads.

Exercise 14 — Square-Number Producer/Consumer

Create a class to store the following array of integers: `int[] numbers = {10, 25, 36, 49, 64};`

Create a thread class that takes each integer from the array and calculates its square, then sends the calculated value to a second thread class for display.

Create a second thread class that displays the squared numbers received from the first thread.

Create a test class to demonstrate the two thread classes above.

Note: use `synchronized`, `wait`, and `notify` to coordinate the data hand-off between the threads.

Exercise 15 — Book Genre Counter (Multithreaded)

Build a `Map<String, Integer>` to store the number of books per genre, for example: - Novel: 10 - Science: 7 - Literature: 12

- **First thread:** every second, randomly picks a genre and increases its count by 1, then displays the chosen genre along with its updated count.
- **Second thread:** tracks the total number of books added every second and displays the running total for the library.

Write a `main()` method to run these two threads, ensuring they operate synchronously and display accurate data.

Exercise 16 — Product Stock In/Out Counter (Multithreaded)

Create a `Map<String, Integer>` to store the quantity of products received and shipped out each hour. For example: - Phone: 20 received, 5 shipped - TV: 15 received, 3 shipped

- **First thread:** every second, randomly picks a product and increases its received quantity by 1, then displays the product and its updated quantity.
- **Second thread:** synchronized with the first thread, randomly picks a different product to update its shipped quantity (decreasing it by 1) and displays the quantity after shipping.

Write a `main()` method to run these two threads, ensuring the data updates accurately and stays synchronized.

Exercise 17 — Hotel Room Booking Counter (Multithreaded)

Create a `Map<String, Integer>` to store the number of rooms booked and returned each hour. For example: - Hotel A: 20 rooms booked, 5 rooms returned. - Hotel B: 15 rooms booked, 3 rooms returned.

Requirements: 1. **First thread:** every second, randomly picks a hotel and increases its booked-room count by 1, then displays the hotel name and the new booked-room count. 2. **Second thread:** synchronized with the first thread, randomly picks a different hotel to update its returned-room count (decreasing it by 1), then displays the hotel name and the new count.

Ensure the room counts update accurately and stay synchronized. Write a `main()` method to run these two threads.

Exercise 18 — Three-Thread Day-Name Pipeline

Create three threads to perform the following tasks:

1. **Thread 1:** every 2 seconds, this thread generates a random day of the week in English (e.g. “Monday”, “Tuesday”, ...), then randomly converts it to either all-uppercase or all-lowercase (e.g. “MONDAY” or “tuesday”).

2. **Thread 2:** receives the random day from Thread 1, normalizes it to standard capitalization (first letter uppercase, rest lowercase), then displays the corresponding day name in Vietnamese. For example: “Monday” → “Thu Hai”, “Sunday” → “Chu Nhat”.

3. **Thread 3:** combines the results from the two threads above (the English day and the Vietnamese day) and saves them to a log file named `days.log`, in the format:

```
Monday - Thu Hai
Sunday - Chu Nhat
```

Additional requirement: synchronize data between the threads using `synchronized`, `wait`, and `notify`. Write a `main()` method to run and display the results from the threads.

Exercise 19 — Fragrance Name Generator and Translator (Two Threads)

Create a class named `FragranceGenerator` that extends `Thread`. - This thread should, after a delay of one second, generate and display a random fragrance name from a predefined list in French. - The possible fragrances are: “Lavande” (Lavender), “Rose” (Rose), “Vanille” (Vanilla), “Citron” (Lemon), “Jasmin” (Jasmine). - Use `Thread.sleep(1000);` to simulate the one-second delay.

Create a class named `FragranceTranslator` that extends `Thread`. - This thread should retrieve the fragrance generated by the `FragranceGenerator` thread and display the corresponding fragrance name in English. - Ensure that the correct translation is displayed: - “Lavande” -> “Lavender” - “Rose” -> “Rose” - “Vanille” -> “Vanilla” - “Citron” -> “Lemon” - “Jasmin” -> “Jasmine”

Use appropriate synchronization mechanisms to ensure that: - The `FragranceTranslator` thread waits until the `FragranceGenerator` thread has generated and displayed the fragrance in French before attempting to translate and display it in English. - Implement this synchronization using `wait()` and `notify()` methods or other suitable synchronization constructs.

Demonstrating Thread Synchronization in `main()`

Write a `main` method to demonstrate the synchronization between the two threads. - The sequence should be as follows: - Start the `FragranceGenerator` thread. - The `FragranceGenerator` thread generates and displays a random fragrance name in French after a one-second delay. - The `FragranceTranslator` thread then retrieves this fragrance and displays the corresponding English translation. - Ensure that this process happens smoothly without race conditions, and that the main thread waits for both threads to complete before exiting. - Run this sequence 3 times, generating and translating 3 different fragrance names.

Exercise 20 — Freelance Skill and Project Description Matcher (Two Threads)

Create a `Map<String, String>` to store freelancer skills and corresponding project descriptions: - Key: Skill (e.g. “Java”, “Python”) - Value: Project description (e.g. “Develop backend systems”, “AI model training”)

First thread — Random Skill Generator: - Generate a random skill from the Map after a 1-second delay. - Display the random skill in the console.

Second thread — Project Description Display: - Retrieve the skill generated by the first thread. - Display the corresponding project description in the console.

Main method: - Synchronize the threads to ensure proper coordination between the skill generation and project description display. - Demonstrate the threads by running the program and ensuring correct outputs.